

ROBOT PROGRAMMING FRAMEWORKS AND IOT PLATFORMS

Scuola Superiore Sant'Anna

Anno Accademico 2025/2026

Gastone Ciuti



How to Program an IoT System: Functions and Coding (15.12.2025) – SLOT 3c

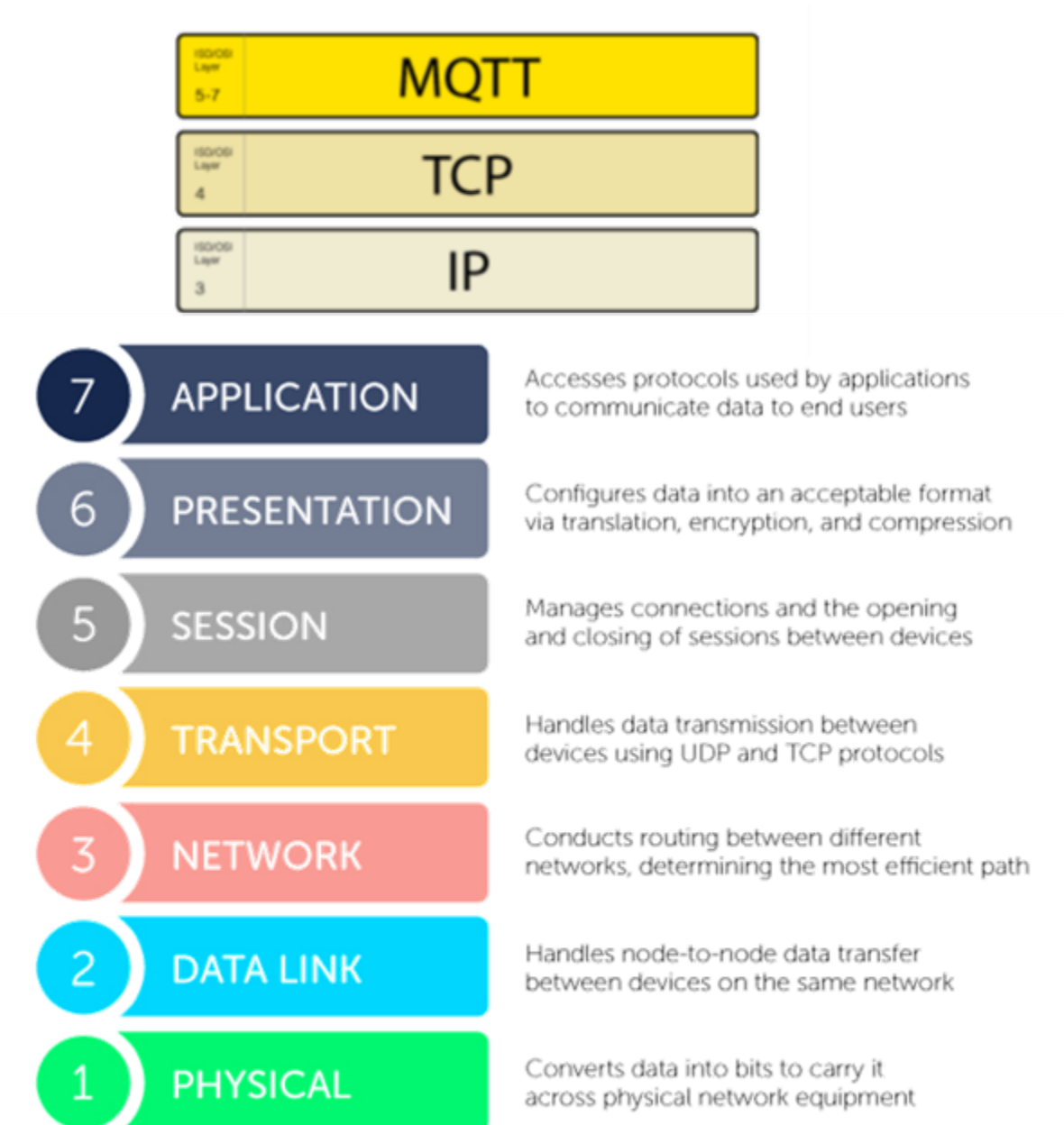
Gastone Ciuti



MQTT MQ Telemetry Transport

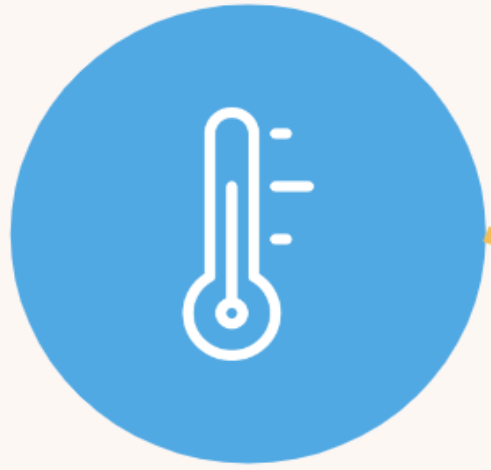
MQTT is a lightweight publish/subscribe messaging protocol designed for M2M (machine to machine) telemetry in low bandwidth environments.

- MQTT stands for **MQ Telemetry Transport** but previously was known as **Message Queuing Telemetry Transport**.
- It was designed by **Andy Stanford-Clark (IBM)** and **Arlen Nipper** in **1999** for connecting Oil Pipeline telemetry systems over satellite.
- MQTT is the **most commonly used messaging protocol** for the Internet of Things (IoT).
- MQTT relies on the TCP transport protocol, and it is a set of rules at the application level that defines how IoT devices can **publish and subscribe to data** over the Internet.
- **Event-driven** and connects devices using the **publish /subscribe (Pub/Sub) pattern**. The sender (**Publisher**) and the receiver (**Subscriber**) communicate via **Topics** and are decoupled from each other.
- The connection clients are handled by the **MQTT broker**. The MQTT broker filters all incoming messages and distributes them correctly to the Subscribers.



MQTT Clients

Ex: Sensors



Publish: **"30°"**
Topic: **"temp"**



Publish: **"65%"**
Topic: **"moisture"**

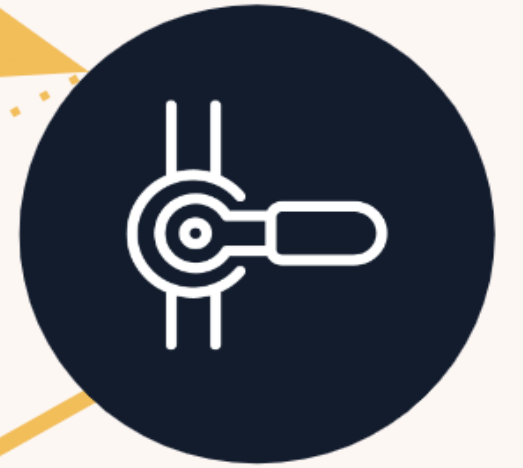


Publish: **"1.8g"**
Topic: **"accel-x"**

**MQTT
Broker**

MQTT Clients

Ex: Valve



Publish: **"30°"**

Subscribed to **"temp"**

Publish: **"closed"**
Topic: **"valve"**

Ex: Cloud Server



Subscribed to **"accel-x"**

Publish: **"1.8g"**

MQTT Why MQTT?

- It requires **minimal resources** since it is lightweight and efficient
- Supports **bi-directional messaging between device and cloud**
- Can **scale to millions of connected devices**
- Supports reliable message delivery through **3 QoS (Quality of Service) levels**
- Works well over **unreliable networks**
- Security enabled, so it works with **TLS (Transport Layer Security)** and **common authentication** protocols for encrypted communication over the internet

MQTT QoS 0



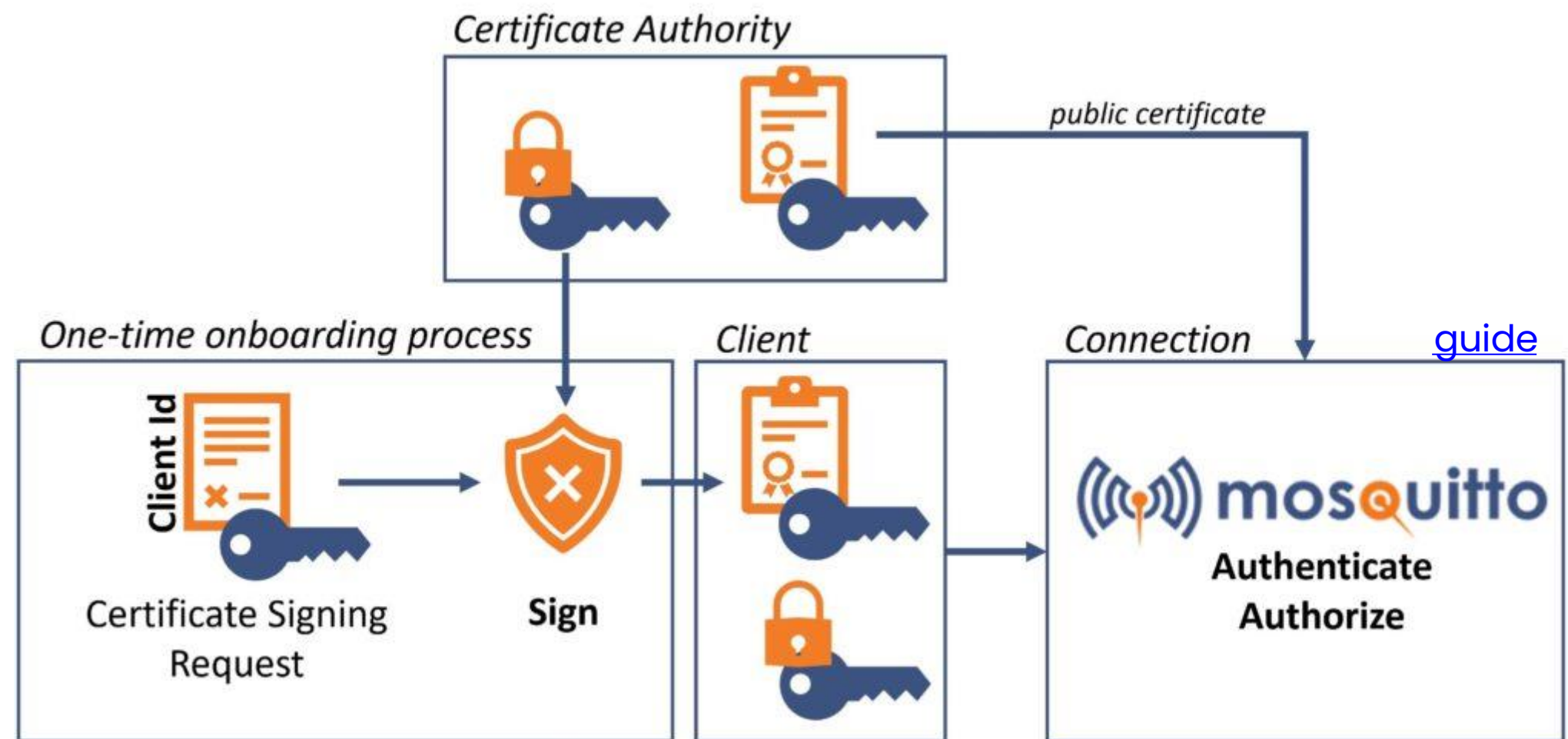
MQTT QoS 1



MQTT QoS 2



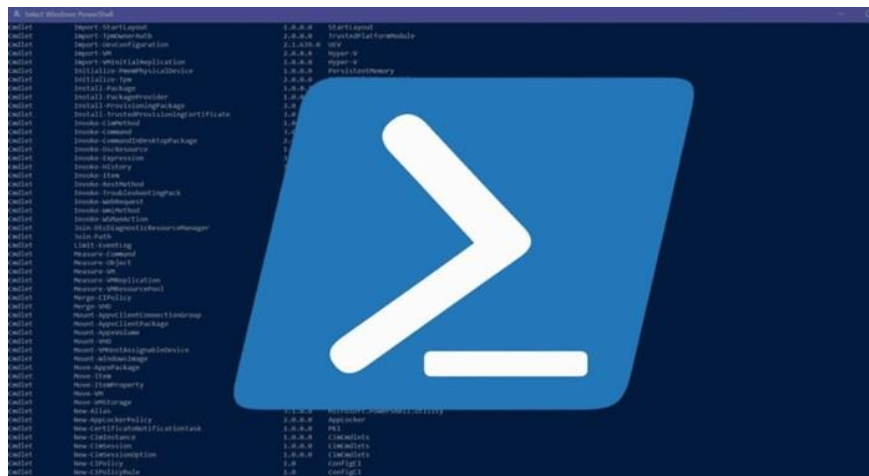
[guide](#)



BROKER mosquitto

Eclipse Mosquitto is an **open-source message broker** that implements the MQTT protocol versions 5.0, 3.1.1 and 3.1. Mosquitto is lightweight and is suitable for use on all devices from low power single board computers to full servers.

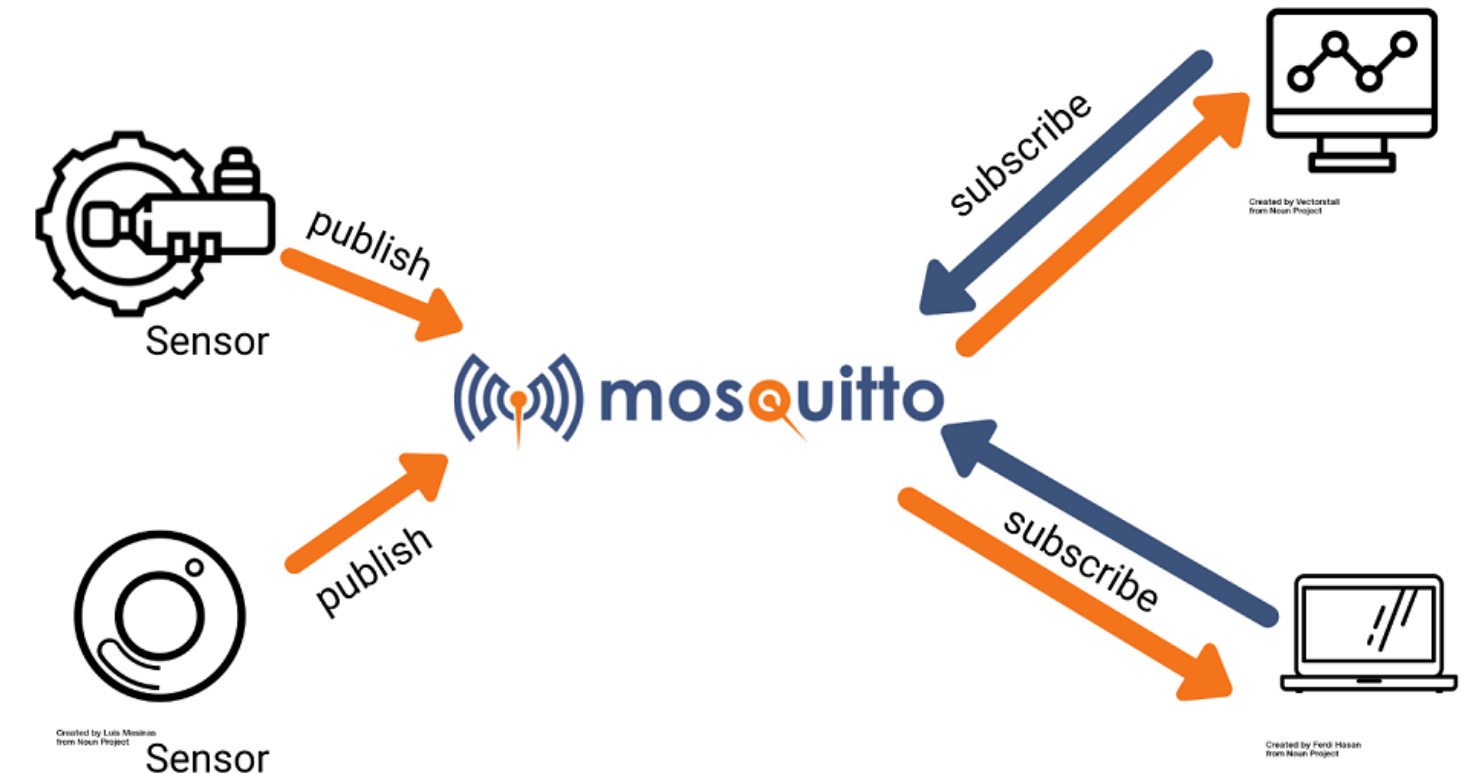
Download «MOSQUITTO» ([link](#))



& "C:\Program Files\mosquitto\mosquitto_sub" -h test.mosquitto.org -p 1883 -t outTopic/message

& "C:\Program Files\mosquitto\mosquitto_pub" -h test.mosquitto.org -p 1883 -t outTopic/message -m "bionics"

Eclipse **Mosquitto**TM
An open-source MQTT broker



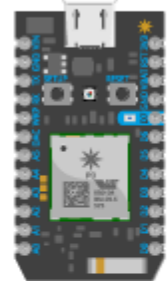


Recipe

if this then that

Trigger

Action



Your Device

[guide](#)

Particle Cloud



HyperText Transfer Protocol (HTTP) at application layer and TCP at transport layer

Web Services



a **web-based automation platform** that enables users to connect different web applications and services to automate tasks and create new functionality.

A **webhook** listens for a **specific event published** by a device. When this event is published, **the webhook triggers a web request to a URL on the web**. The request sent by the webhook can include information about the event as well as any data included when the event was published.

The most common type of webhook request is a **POST**, which is a **method of sending data to another web server**. In the case of Particle webhooks, this would mean sending data from your devices to a third-party web service (such as IFTTT).

Oftentimes, a web server you hit with a **webhook will return data** to you as a result of the request made. When this happens, your **devices can subscribe to a specific event name to receive the response from the web server** and use it in your firmware logic.

Particle strongly recommends **using https (TLS/SSL) for all webhook communication for security**.





[guide](#)

```
{
  "first_name": "John",
  "last_name": "Smith",
  "is_alive": true,
  "age": 27,
  "address": {
    "street_address": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal_code": "10021-3100"
  },
  "phone_numbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [
    "Catherine",
    "Thomas",
    "Trevor"
  ],
  "spouse": null
}
```

{JSON}

JSON (JavaScript Object Notation) is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of **name-value pairs and arrays** (or other serializable values). It is a commonly used data format with diverse uses in electronic data interchange, including that of **web applications with servers**.

JSON is a **language-independent data format**. It was derived from **JavaScript**, but many modern programming languages include code to generate and parse JSON-format data. JSON filenames use the extension .json.



REFERENCES

- Particle website: <https://www.particle.io/>
- Particle store: <https://store.particle.io/products/>
- Particle platform guides: <https://docs.particle.io/>
- Particle ARGON datasheet: <https://docs.particle.io/reference/datasheets/wi-fi/argon-datasheet/>
- Particle set-up: <https://setup.particle.io/>
- Particle «doctor»: <https://docs.particle.io/tools/doctor/>
- Particle reference manual: <https://docs.particle.io/reference/device-os/firmware/>
- Particle Web IDE: <https://build.particle.io/>
- Github repository: <https://github.com/particle-iot/argon>
- Seedstudio website: <https://www.seeedstudio.com/>





gastone.ciuti@santannapisa.it

